Hyperledger Fabric: análise do asset-transfer-basic

O que é o Hyperledger Fabric

O Hyperledger Fabric é uma plataforma de blockchain permissionada e open source, mantida pela Linux Foundation: só participa quem tem identidade emitida por uma organização da rede, e não há criptomoeda nem mineração.

Cada organização opera **peers**, que guardam o ledger e executam os smart contracts, chamados de **chaincode**. Um **orderer** ordena as transações em blocos. O ledger tem duas partes: o **world state** (estado atual, em chave/valor) e a **blockchain** (histórico imutável).

Uma gravação segue o fluxo execute → order → validate: os peers simulam o chaincode e assinam o resultado (endosso), o orderer monta o bloco e cada peer valida antes de gravar. Uma consulta executa num peer só e não gera bloco. O cliente acessa a rede pela API do **Fabric Gateway**, um serviço do peer que coordena essas etapas.

Os três arquivos analisados vêm do sample `asset-transfer-basic`, do repositório `fabric-samples`; ele pode ser visitado em https://github.com/hyperledger/fabric-samples.

smartcontract.go

O contrato (`chaincode-go/chaincode/smartcontract.go`) concentra a regra de negócio: cada método exportado vira uma transação.

Pacote e imports

```
package chaincode

import (
	"encoding/json"
	"fmt"

	"github.com/hyperledger/fabric-contract-api-go/v2/contractapi"
)
```

- `package chaincode`: nome do pacote. É importado pelo `main` do chaincode.
- `encoding/json`: converte structs em JSON e de volta.
- `fmt`: formatação de texto, usada nas mensagens de erro.
- `contractapi`: API de contrato do Fabric para Go.
- **Go:** a convenção separa os imports da biblioteca padrão dos externos com uma linha em branco. Um import não usado é erro de compilação.

Tipos

```
type SmartContract struct {
	contractapi.Contract
}
```

- Define o tipo do contrato.
- `contractapi.Contract`, sem nome de campo, é um campo embutido: `SmartContract` recebe os métodos de `Contract` e vira um contrato válido para o Fabric.
- **Go:** não existem classes nem herança. O embedding é composição: os métodos do tipo embutido passam a ser chamáveis direto no tipo externo.

```
type Asset struct {
	AppraisedValue int    `json:"AppraisedValue"`
	Color          string `json:"Color"`
	ID             string `json:"ID"`
	Owner          string `json:"Owner"`
	Size           int    `json:"Size"`
}
```

- Cada linha declara nome, tipo e tag de um campo.
- A tag `json:"..."` define o nome da chave no JSON.
- Os campos estão em ordem alfabética por determinismo: `json.Marshal` segue a ordem de declaração, e peers em linguagens diferentes precisam gerar os mesmos bytes para o endosso coincidir.
- **Go:** inicial maiúscula torna um nome exportado (público). `json.Marshal` só enxerga campos exportados. A mesma regra decide quais métodos do contrato viram transações.

InitLedger

```
func (s *SmartContract) InitLedger(ctx contractapi.TransactionContextInterface) error
{
	assets := []Asset{
		{ID: "asset1", Color: "blue", Size: 5, Owner: "Tomoko", AppraisedValue:
300},
		{ID: "asset2", Color: "red", Size: 5, Owner: "Brad", AppraisedValue:
400},
		{ID: "asset3", Color: "green", Size: 10, Owner: "Jin Soo",
AppraisedValue: 500},
		{ID: "asset4", Color: "yellow", Size: 10, Owner: "Max", AppraisedValue:
600},
		{ID: "asset5", Color: "black", Size: 15, Owner: "Adriana",
AppraisedValue: 700},
		{ID: "asset6", Color: "white", Size: 15, Owner: "Michel",
AppraisedValue: 800},
	}

	for _, asset := range assets {
		assetJSON, err := json.Marshal(asset)
		if err != nil {
			return err
		}

		err = ctx.GetStub().PutState(asset.ID, assetJSON)
		if err != nil {
			return fmt.Errorf("failed to put to world state. %w", err)
		}
	}
```

```
	return nil
}
```

- `(s *SmartContract)`: o receiver torna a função um método do contrato.
- `ctx`: contexto da transação, injetado pelo Fabric. `ctx.GetStub()` acessa o ledger; `ctx.GetClientIdentity()`, a identidade de quem chamou.
- `error`: tipo de retorno; `nil` significa sucesso.
- `assets := []Asset{...}`: cria uma lista com os 6 ativos iniciais, cada um preenchido pelo nome dos campos.
- `for _, asset := range assets`: percorre a lista; o `_` descarta o índice.
- `json.Marshal(asset)`: converte o ativo em JSON (`[]byte`).
- `PutState(asset.ID, assetJSON)`: grava o par chave/valor. Na prática, só registra a escrita no write set da simulação; a gravação real ocorre após ordenação e validação.
- `fmt.Errorf("... %w", err)`: cria um erro novo que embrulha o original, preservando a causa.
- **Go:** `:=` declara e infere o tipo; `=` reatribui uma variável que já existe (o `err` do `PutState`). Erros são valores retornados, não exceções, daí o `if err != nil` repetido em todo o arquivo.
- **Go:** o receiver é ponteiro (`*SmartContract`), então o método recebe uma referência ao contrato, não uma cópia.

CreateAsset

```
func (s *SmartContract) CreateAsset(ctx contractapi.TransactionContextInterface, id
string, color string, size int, owner string, appraisedValue int) error {
	exists, err := s.AssetExists(ctx, id)
	if err != nil {
		return err
	}
	if exists {
		return fmt.Errorf("the asset %s already exists", id)
	}

	asset := Asset{
		ID:             id,
		Color:          color,
		Size:           size,
		Owner:          owner,
		AppraisedValue: appraisedValue,
	}
	assetJSON, err := json.Marshal(asset)
	if err != nil {
		return err
	}

	return ctx.GetStub().PutState(id, assetJSON)
}
```

- Os parâmetros chegam do cliente como strings; a `contractapi` converte para os tipos declarados (`"5"` → `int`).
- `s.AssetExists(ctx, id)`: chama outro método do próprio contrato pelo receiver `s`.
- `if exists`: recusa ID duplicado.
- `asset := Asset{...}`: monta o ativo com os valores recebidos.
- `return ctx.GetStub().PutState(...)`: devolve direto o erro do `PutState`, ou `nil`.

- Duas criações simultâneas do mesmo ID passam pela checagem na simulação; a validação MVCC no commit invalida a segunda.
- **Go:** parâmetros do mesmo tipo podem ser agrupados (`id, color string`); aqui cada um foi declarado separado. Funções podem retornar vários valores, como `(bool, error)` no `AssetExists`.

ReadAsset

```
func (s *SmartContract) ReadAsset(ctx contractapi.TransactionContextInterface, id
string) (*Asset, error) {
	assetJSON, err := ctx.GetStub().GetState(id)
	if err != nil {
		return nil, fmt.Errorf("failed to read from world state: %w", err)
	}
	if assetJSON == nil {
		return nil, fmt.Errorf("the asset %s does not exist", id)
	}

	var asset Asset
	err = json.Unmarshal(assetJSON, &asset)
	if err != nil {
		return nil, err
	}

	return &asset, nil
}
```

- `(*Asset, error)`: retorna um ponteiro para o ativo e um erro.
- `GetState(id)`: lê o valor já commitado no world state.
- `if assetJSON == nil`: chave inexistente não gera erro no `GetState`, só retorna `nil`. Por isso a checagem à parte.
- `var asset Asset`: cria a struct com valores zerados.
- `json.Unmarshal(assetJSON, &asset)`: preenche a struct a partir do JSON.
- `return &asset, nil`: devolve o endereço da struct.
- **Go:** toda variável nasce com o valor zero do seu tipo (`0`, `""`, `nil`). `&` obtém o endereço de uma variável; é assim que o `Unmarshal` consegue alterá-la.
- **Go:** retornar o endereço de uma variável local é seguro. O compilador detecta isso (escape analysis) e a aloca no heap.

UpdateAsset

```
func (s *SmartContract) UpdateAsset(ctx contractapi.TransactionContextInterface, id
string, color string, size int, owner string, appraisedValue int) error {
	exists, err := s.AssetExists(ctx, id)
	if err != nil {
		return err
	}
	if !exists {
		return fmt.Errorf("the asset %s does not exist", id)
	}
```

```
	// overwriting original asset with new asset
	asset := Asset{
		ID:             id,
		Color:          color,
		Size:           size,
		Owner:          owner,
		AppraisedValue: appraisedValue,
	}
	assetJSON, err := json.Marshal(asset)
	if err != nil {
		return err
	}

	return ctx.GetStub().PutState(id, assetJSON)
}
```

- Mesma estrutura do `CreateAsset`, com a checagem invertida: `!exists` exige que o ativo exista.
- A mensagem `the asset %s does not exist` é a que o cliente recebe no teste de erro com o `asset70`.
- O ativo é sobrescrito inteiro: o cliente precisa enviar todos os campos.
- **Go:** `!` é a negação lógica; `if` não usa parênteses, mas as chaves são obrigatórias.

DeleteAsset

```
func (s *SmartContract) DeleteAsset(ctx contractapi.TransactionContextInterface, id
string) error {
	exists, err := s.AssetExists(ctx, id)
	if err != nil {
		return err
	}
	if !exists {
		return fmt.Errorf("the asset %s does not exist", id)
	}

	return ctx.GetStub().DelState(id)
}
```

- Exige que o ativo exista e o remove com `DelState`.
- A chave sai do world state, mas as transações anteriores continuam nos blocos: o histórico é imutável.

AssetExists

```
func (s *SmartContract) AssetExists(ctx contractapi.TransactionContextInterface, id
string) (bool, error) {
	assetJSON, err := ctx.GetStub().GetState(id)
	if err != nil {
		return false, fmt.Errorf("failed to read from world state: %w", err)
	}

	return assetJSON != nil, nil
}
```

- Lê a chave e retorna `true` se ela tem valor.
- `return assetJSON != nil, nil`: o primeiro valor é o resultado da comparação.

- Por ser exportada, também é uma transação pública, não só uma função auxiliar.

TransferAsset

```
func (s *SmartContract) TransferAsset(ctx contractapi.TransactionContextInterface, id
string, newOwner string) (string, error) {
	asset, err := s.ReadAsset(ctx, id)
	if err != nil {
		return "", err
	}

	oldOwner := asset.Owner
	asset.Owner = newOwner

	assetJSON, err := json.Marshal(asset)
	if err != nil {
		return "", err
	}

	err = ctx.GetStub().PutState(id, assetJSON)
	if err != nil {
		return "", err
	}

	return oldOwner, nil
}
```

- Reaproveita o `ReadAsset`, que já trata o ativo inexistente.
- Guarda o dono antigo, troca pelo novo e grava.
- Retorna o dono antigo; o cliente usa esse valor na mensagem de transferência.
- Não há validação de propriedade: qualquer identidade da rede pode transferir qualquer ativo.
- **Go:** `asset` é um ponteiro, e `asset.Owner` acessa o campo sem precisar escrever `(*asset).Owner`. Em caso de erro, retorna-se o valor zero do tipo (`""`) junto com o erro.

GetAllAssets

```
func (s *SmartContract) GetAllAssets(ctx contractapi.TransactionContextInterface)
([]*Asset, error) {
	// range query with empty string for startKey and endKey does an
	// open-ended query of all assets in the chaincode namespace.
	resultsIterator, err := ctx.GetStub().GetStateByRange("", "")
	if err != nil {
		return nil, err
	}
	defer resultsIterator.Close()

	var assets []*Asset
	for resultsIterator.HasNext() {
		queryResponse, err := resultsIterator.Next()
		if err != nil {
			return nil, err
		}
```

```
            var asset Asset
            err = json.Unmarshal(queryResponse.GetValue(), &asset)
            if err != nil {
                  return nil, err
            }
            assets = append(assets, &asset)
      }

      return assets, nil
}
```

- `GetStateByRange("", "")`: intervalo aberto, ou seja, todas as chaves do namespace deste chaincode.
- O resultado é um iterador, que entrega um item por vez.
- `defer resultsIterator.Close()`: agenda o fechamento para o fim da função, por qualquer caminho de saída.
- `for resultsIterator.HasNext()`: repete enquanto houver itens.
- `Next()`: devolve o próximo par chave/valor; `GetValue()` pega o JSON.
- `append(assets, &asset)`: acrescenta o ativo à lista.
- A `contractapi` serializa o `[]*Asset` retornado em JSON automaticamente.
- Com LevelDB só há consultas por chave e por intervalo; consultas por conteúdo exigem CouchDB.
- **Go:** `for` é o único laço da linguagem; `for condição {}` faz o papel do `while`. `defer` executa ao sair da função, e vários `defer` rodam na ordem inversa.
- **Go:** `append` pode realocar a lista, por isso o resultado é sempre reatribuído. `var asset Asset` dentro do laço cria uma variável nova a cada volta, então cada `&asset` aponta para um ativo diferente.

chaincode-go/assetTransfer.go

O `main` do chaincode registra o contrato e o conecta ao peer.

```
package main

import (
      "log"

      "github.com/hyperledger/fabric-contract-api-go/v2/contractapi"
      "github.com/hyperledger/fabric-samples/asset-transfer-basic/chaincode-go/
chaincode"
)
```

- `package main`: indica um programa executável. O peer compila esse binário e o roda num container próprio.
- `log`: biblioteca padrão de logs.
- `contractapi`: a mesma API usada no contrato.
- O último import é o pacote `chaincode`, ou seja, o `smartcontract.go`.
- **Go:** o caminho de um import do próprio projeto é o nome do módulo declarado no `go.mod` mais a subpasta. O pacote é referenciado pelo último trecho do caminho (`chaincode`).

```
func main() {
	assetChaincode, err := contractapi.NewChaincode(&chaincode.SmartContract{})
	if err != nil {
		log.Panicf("Error creating asset-transfer-basic chaincode: %v", err)
	}
```

- `main()`: ponto de entrada do programa.
- `chaincode.SmartContract{}`: cria uma instância vazia do contrato; o `&` passa seu endereço.
- `NewChaincode`: inspeciona o contrato e registra cada método exportado como transação, com a conversão de argumentos e retornos.
- `log.Panicf`: registra a mensagem e interrompe o programa. `%v` imprime o valor do erro.
- **Go:** `&Tipo{}` cria a struct e já devolve seu endereço numa expressão só. O ponteiro é necessário porque os métodos do contrato têm receiver `*SmartContract`.
- **Go:** a inspeção de métodos em tempo de execução usa reflection (pacote `reflect`), recurso que a `contractapi` usa por baixo.

```
	if err := assetChaincode.Start(); err != nil {
		log.Panicf("Error starting asset-transfer-basic chaincode: %v", err)
	}
}
```

- `Start()`: conecta o chaincode ao peer via gRPC, na porta 7052, e fica atendendo chamadas. Só retorna em caso de erro.
- **Go:** `if x := ...; condição` declara uma variável que só existe dentro do `if` e do `else`. Isso evita que o `err` vaze para o resto da função.

application-gateway-go/assetTransfer.go

O cliente se conecta ao peer da Org1 pela API do Fabric Gateway e executa seis transações em sequência.

Pacote e imports

```
package main

import (
	"bytes"
	"context"
	"crypto/x509"
	"encoding/json"
	"errors"
	"fmt"
	"os"
	"path"
	"time"

	"github.com/hyperledger/fabric-gateway/pkg/client"
	"github.com/hyperledger/fabric-gateway/pkg/hash"
	"github.com/hyperledger/fabric-gateway/pkg/identity"
	"google.golang.org/grpc"
	"google.golang.org/grpc/credentials"
```

```
	"google.golang.org/grpc/status"
)
```

- `package main`: programa executável, rodado com `go run .`.
- `bytes` e `encoding/json`: formatação do JSON de resposta.
- `context` e `errors`: classificação dos erros.
- `crypto/x509`: certificados usados no TLS.
- `os`, `path` e `time`: arquivos, variáveis de ambiente, timeouts e geração do ID.
- `fabric-gateway/pkg/...`: a API do Fabric Gateway (conexão, transações, hash e identidade).
- `google.golang.org/grpc/...`: o gRPC, protocolo de rede usado pelo Fabric.

Constantes e variáveis

```
const (
	mspID        = "Org1MSP"
	cryptoPath   =
"../../test-network/organizations/peerOrganizations/org1.example.com"
	certPath     = cryptoPath + "/users/User1@org1.example.com/msp/signcerts"
	keyPath      = cryptoPath + "/users/User1@org1.example.com/msp/keystore"
	tlsCertPath  = cryptoPath + "/peers/peer0.org1.example.com/tls/ca.crt"
	peerEndpoint = "dns:///localhost:7051"
	gatewayPeer  = "peer0.org1.example.com"
)

var now = time.Now()
var assetID = fmt.Sprintf("asset%d", now.Unix()*1e3+int64(now.Nanosecond())/1e6)
```

- `mspID`: o cliente se identifica como membro da Org1.
- `cryptoPath`: pasta com os certificados da Org1, em caminho relativo. O programa só funciona rodando de dentro de `application-gateway-go`.
- `certPath` (`signcerts`): certificado público do `User1`, que diz quem ele é.
- `keyPath` (`keystore`): chave privada do `User1`, que assina as transações.
- `tlsCertPath`: certificado da CA de TLS do peer, para confirmar que o servidor é legítimo.
- `peerEndpoint`: endereço do peer; `dns:///` indica ao gRPC como resolver o nome.
- `gatewayPeer`: nome que consta no certificado TLS do peer.
- `assetID`: "asset" + milissegundos desde 1970 (segundos × 1000 + nanossegundos ÷ 1.000.000), para gerar um ID novo a cada execução.
- **Go:** `const ( ... )` agrupa constantes, resolvidas em tempo de compilação, e uma constante pode ser montada a partir de outra. `var` fora de funções cria variáveis do pacote, inicializadas antes da `main()`.
- **Go:** `1e3` é uma constante numérica sem tipo, que se adapta ao `int64` da conta. Já `now.Nanosecond()` retorna `int` e precisa de conversão explícita com `int64(...)`, porque o Go não converte tipos numéricos sozinho.

main

```
func main() {
	// The gRPC client connection should be shared by all Gateway connections to
this endpoint
	clientConnection := newGrpcConnection()
	defer clientConnection.Close()
```

```
	id := newIdentity()
	sign := newSign()
```

- Cria a conexão gRPC, a identidade e a função de assinatura.
- `defer clientConnection.Close()`: fecha a conexão quando a `main` terminar.

```
	// Create a Gateway connection for a specific client identity
	gw, err := client.Connect(
		id,
		client.WithSign(sign),
		client.WithHash(hash.SHA256),
		client.WithClientConnection(clientConnection),
		// Default timeouts for different gRPC calls
		client.WithEvaluateTimeout(5*time.Second),
		client.WithEndorseTimeout(15*time.Second),
		client.WithSubmitTimeout(5*time.Second),
		client.WithCommitStatusTimeout(1*time.Minute),
	)
	if err != nil {
		panic(err)
	}
	defer gw.Close()
```

- `client.Connect`: abre a conexão com o Gateway usando a identidade do `User1`.
- `WithSign` e `WithHash(hash.SHA256)`: cada transação tem o hash SHA-256 calculado e assinado com a chave privada.
- `WithClientConnection`: reaproveita a conexão gRPC criada antes.
- Os quatro timeouts valem por etapa: consulta 5 s, endosso 15 s, envio ao orderer 5 s e espera do commit 1 min.
- `panic(err)`: aborta o programa se a conexão falhar.
- **Go:** o padrão de "opções funcionais" (`client.WithX(...)`) passa configurações opcionais como funções, já que a linguagem não tem parâmetros nomeados nem valores padrão.
- **Go:** a vírgula após o último argumento é obrigatória quando o fechamento `)` fica em outra linha.

```
	// Override default values for chaincode and channel name as they may differ
in testing contexts.
	chaincodeName := "basic"
	if ccname := os.Getenv("CHAINCODE_NAME"); ccname != "" {
		chaincodeName = ccname
	}

	channelName := "mychannel"
	if cname := os.Getenv("CHANNEL_NAME"); cname != "" {
		channelName = cname
	}

	network := gw.GetNetwork(channelName)
	contract := network.GetContract(chaincodeName)

	initLedger(contract)
	getAllAssets(contract)
	createAsset(contract)
	readAssetByID(contract)
	transferAssetAsync(contract)
```

```
	exampleErrorHandling(contract)
}
```

- Usa `basic` e `mychannel` por padrão, ou os valores das variáveis de ambiente `CHAINCODE_NAME` e `CHANNEL_NAME`.
- `GetNetwork` e `GetContract`: descem a hierarquia Gateway → canal → contrato.
- As seis funções seguintes executam as transações em sequência.
- **Go:** `os.Getenv` retorna `""` quando a variável não existe; não há `null` para strings.

newGrpcConnection

```
func newGrpcConnection() *grpc.ClientConn {
	certificatePEM, err := os.ReadFile(tlsCertPath)
	if err != nil {
		panic(fmt.Errorf("failed to read TLS certifcate file: %w", err))
	}

	certificate, err := identity.CertificateFromPEM(certificatePEM)
	if err != nil {
		panic(err)
	}

	certPool := x509.NewCertPool()
	certPool.AddCert(certificate)
	transportCredentials := credentials.NewClientTLSFromCert(certPool,
gatewayPeer)

	connection, err := grpc.NewClient(peerEndpoint,
grpc.WithTransportCredentials(transportCredentials))
	if err != nil {
		panic(fmt.Errorf("failed to create gRPC connection: %w", err))
	}

	return connection
}
```

- `os.ReadFile`: lê o certificado TLS em formato PEM (texto com `-----BEGIN CERTIFICATE-----`).
- `CertificateFromPEM`: converte o texto num objeto de certificado.
- `NewCertPool` e `AddCert`: montam a lista de CAs confiáveis. A conexão só aceita servidores cujo certificado foi assinado por uma delas.
- `NewClientTLSFromCert(certPool, gatewayPeer)`: o segundo argumento sobrescreve o nome esperado do servidor. A conexão vai para `localhost`, mas o certificado foi emitido para `peer0.org1.example.com`.
- `grpc.NewClient`: cria o cliente gRPC com TLS; a conexão de rede só é aberta na primeira chamada.
- **Go:** `panic` interrompe o programa na hora. Num sample isso é aceitável; numa aplicação real, o idiomático é retornar o erro a quem chamou.

newIdentity

```
func newIdentity() *identity.X509Identity {
	certificatePEM, err := readFirstFile(certPath)
	if err != nil {
		panic(fmt.Errorf("failed to read certificate file: %w", err))
	}

	certificate, err := identity.CertificateFromPEM(certificatePEM)
	if err != nil {
		panic(err)
	}

	id, err := identity.NewX509Identity(mspID, certificate)
	if err != nil {
		panic(err)
	}

	return id
}
```

- Lê o certificado do `User1` e o junta ao `mspID`.
- O resultado é a identidade que vai dentro de cada transação; no chaincode, ela aparece em `ctx.GetClientIdentity()`.
- Diferente do TLS, que protege só a conexão, essa identidade fica gravada no bloco junto com a assinatura.

newSign

```
func newSign() identity.Sign {
	privateKeyPEM, err := readFirstFile(keyPath)
	if err != nil {
		panic(fmt.Errorf("failed to read private key file: %w", err))
	}

	privateKey, err := identity.PrivateKeyFromPEM(privateKeyPEM)
	if err != nil {
		panic(err)
	}

	sign, err := identity.NewPrivateKeySign(privateKey)
	if err != nil {
		panic(err)
	}

	return sign
}
```

- Lê a chave privada e retorna uma função que assina dados com ela.
- A chave nunca sai do cliente; só as assinaturas vão pela rede.
- **Go:** funções são valores: `identity.Sign` é um tipo de função, que pode ser guardado numa variável, passado e retornado como qualquer outro valor.

readFirstFile

```
func readFirstFile(dirPath string) ([]byte, error) {
	dir, err := os.Open(dirPath)
	if err != nil {
		return nil, err
	}

	fileNames, err := dir.Readdirnames(1)
	if err != nil {
		return nil, err
	}

	return os.ReadFile(path.Join(dirPath, fileNames[0]))
}
```

- Abre a pasta, pega o nome do primeiro arquivo e o lê.
- Necessário porque o arquivo da chave privada tem um nome aleatório, gerado pela CA.
- Falta um `defer dir.Close()`; a pasta fica aberta até o programa terminar, sem efeito prático aqui.
- **Go:** função com inicial minúscula é privada ao pacote. `path.Join` junta os trechos do caminho com barras (/); `fileNames[0]` é o primeiro item da lista.

initLedger e createAsset

```
func initLedger(contract *client.Contract) {
	fmt.Printf("\n--> Submit Transaction: InitLedger, function creates the initial set of assets on the ledger \n")

	_, err := contract.SubmitTransaction("InitLedger")
	if err != nil {
		panic(fmt.Errorf("failed to submit transaction: %w", err))
	}

	fmt.Printf("*** Transaction committed successfully\n")
}

func createAsset(contract *client.Contract) {
	fmt.Printf("\n--> Submit Transaction: CreateAsset, creates new asset with ID, Color, Size, Owner and AppraisedValue arguments \n")

	_, err := contract.SubmitTransaction("CreateAsset", assetID, "yellow", "5", "Tom", "1300")
	if err != nil {
		panic(fmt.Errorf("failed to submit transaction: %w", err))
	}

	fmt.Printf("*** Transaction committed successfully\n")
}
```

- `SubmitTransaction`: faz o ciclo completo e síncrono (endosso, envio ao orderer, espera do commit). Só retorna com a transação gravada no bloco.
- O primeiro argumento é o nome da função do contrato; os demais são seus parâmetros, sempre como strings.

- O retorno do chaincode é descartado com `_`.
- **Go:** `SubmitTransaction` aceita qualquer quantidade de argumentos (função variádica, declarada com `...string`).

getAllAssets e readAssetByID

```
func getAllAssets(contract *client.Contract) {
	fmt.Println("\n--> Evaluate Transaction: GetAllAssets, function returns all
the current assets on the ledger")

	evaluateResult, err := contract.EvaluateTransaction("GetAllAssets")
	if err != nil {
		panic(fmt.Errorf("failed to evaluate transaction: %w", err))
	}
	result := formatJSON(evaluateResult)

	fmt.Printf("*** Result:%s\n", result)
}

func readAssetByID(contract *client.Contract) {
	fmt.Printf("\n--> Evaluate Transaction: ReadAsset, function returns asset
attributes\n")

	evaluateResult, err := contract.EvaluateTransaction("ReadAsset", assetID)
	if err != nil {
		panic(fmt.Errorf("failed to evaluate transaction: %w", err))
	}
	result := formatJSON(evaluateResult)

	fmt.Printf("*** Result:%s\n", result)
}
```

- `EvaluateTransaction`: executa o chaincode num peer só, sem orderer e sem bloco.
- O retorno chega como `[]byte` (o JSON do chaincode) e é formatado para exibição.
- O `ReadAsset` roda antes da transferência, por isso mostra `Tom` como dono.
- **Go:** `Println` adiciona quebra de linha no fim; `Printf` usa verbos de formatação (`%s` para texto, `%d` para inteiro, `%v` para qualquer valor).

transferAssetAsync

```
func transferAssetAsync(contract *client.Contract) {
	fmt.Printf("\n--> Async Submit Transaction: TransferAsset, updates existing
asset owner")

	submitResult, commit, err := contract.SubmitAsync("TransferAsset",
client.WithArguments(assetID, "Mark"))
	if err != nil {
		panic(fmt.Errorf("failed to submit transaction asynchronously: %w",
err))
	}

	fmt.Printf("\n*** Successfully submitted transaction to transfer ownership
from %s to Mark. \n", string(submitResult))
```

```
	fmt.Println("*** Waiting for transaction commit.")

	if commitStatus, err := commit.Status(); err != nil {
		panic(fmt.Errorf("failed to get commit status: %w", err))
	} else if !commitStatus.Successful {
		panic(fmt.Errorf("transaction %s failed to commit with status: %d",
commitStatus.TransactionID, int32(commitStatus.Code)))
	}

	fmt.Printf("*** Transaction committed successfully\n")
}
```

- `SubmitAsync`: retorna assim que o orderer recebe a transação, sem esperar o commit.
- `submitResult`: retorno do chaincode, o dono antigo (`Tom`).
- `commit`: objeto para consultar o commit depois; `commit.Status()` bloqueia até ele acontecer.
- `!commitStatus.Successful`: a transação foi ordenada, mas invalidada na validação (por exemplo, conflito MVCC). Ela entra no bloco marcada como inválida e não altera o estado.
- O modo assíncrono permite reagir antes do commit, como atualizar uma tela.
- **Go:** `string(submitResult)` converte `[]byte` em texto. O `if ... else if` reaproveita as variáveis declaradas no `if` inicial (`commitStatus`, `err`).

exampleErrorHandling

```
func exampleErrorHandling(contract *client.Contract) {
	fmt.Println("\n--> Submit Transaction: UpdateAsset asset70, asset70 does not
exist and should return an error")

	_, err := contract.SubmitTransaction("UpdateAsset", "asset70", "blue", "5",
"Tomoko", "300")
	if err == nil {
		panic("******** FAILED to return an error")
	}

	fmt.Println("*** Successfully caught the error:")

	var commitStatusErr *client.CommitStatusError
	var transactionErr *client.TransactionError

	if errors.As(err, &commitStatusErr) {
		if errors.Is(err, context.DeadlineExceeded) {
			fmt.Printf("Timeout waiting for transaction %s commit status:
%s\n", commitStatusErr.TransactionID, commitStatusErr)
		} else {
			fmt.Printf("Error obtaining commit status for transaction %s with
gRPC status %v: %s\n", commitStatusErr.TransactionID, status.Code(commitStatusErr),
commitStatusErr)
		}
	} else if errors.As(err, &transactionErr) {
		// The error could be an EndorseError, SubmitError or CommitError.
		fmt.Println(err)
		fmt.Printf("TransactionID: %s\n", transactionErr.TransactionID)
	} else {
		panic(fmt.Errorf("unexpected error type %T: %w", err, err))
```

```
	}
}
```

- Força um erro: o `asset70` não existe, então o `UpdateAsset` do contrato falha no endosso. Se não houver erro, aí sim algo está errado.
- `CommitStatusError`: não foi possível saber o resultado do commit, por timeout (`context.DeadlineExceeded`) ou falha de rede.
- `TransactionError`: a transação falhou numa etapa, como `EndorseError`, `SubmitError` ou `CommitError`. É o caso deste teste, que imprime o erro e o `TransactionID`.
- O comentário original do sample fala em "wrong number of arguments", mas está desatualizado: o erro vem do ativo inexistente.
- **Go:** `errors.As` verifica se o erro, ou algum erro embrulhado com `%w`, é de um determinado tipo e, se for, preenche a variável. `errors.Is` compara com um valor específico.
- **Go:** `%T` imprime o tipo de uma variável, útil para depurar erros inesperados.

formatJSON

```
func formatJSON(data []byte) string {
	var prettyJSON bytes.Buffer
	if err := json.Indent(&prettyJSON, data, "", "  "); err != nil {
		panic(fmt.Errorf("failed to parse JSON: %w", err))
	}
	return prettyJSON.String()
}
```

- Reescreve o JSON com indentação de 2 espaços e devolve como texto. Serve só para exibição.
- **Go:** `bytes.Buffer` funciona sem inicialização: o valor zero já é um buffer vazio e pronto para uso.

Referências

- hyperledger/fabric-samples (GitHub)
- asset-transfer-basic (README)